

JSP3: Sentiment and Entity Analysis

Jakub Pokorný ([pokoja12](mailto:pokoja12@upol.cz))
pokoja12@upol.cz

Bára Havlíčková
bara.havlickova01@upol.cz

29 January 2026

1 Introduction

This work presents an analysis of literary text, focusing on measuring sentiment and identifying named entities through natural language processing techniques. Sentiment is evaluated using three complementary methods: VADER for context-sensitive sentence-level scoring, a WordNet-based lexicon approach for sense-aware word-level sentiment, and TextBlob for sentence-level polarity analysis. The average sentiment for each method is calculated over both text chapters and textual blocks of defined length, independent of chapters, and the results are visualized.

In addition, the code identifies and tracks named entities throughout the text, recording their positions and categorizing them by type. This structured representation enables the mapping of characters, locations, times, and other entities, with the results also visualized.

2 Background

This work relies on several natural language processing resources to perform sentiment and entity analysis on literary texts. For sentiment analysis, we use lexicon-based and rule-based approaches, complemented by Python libraries. For entity analysis, we employ NLP models capable of recognizing named entities and their positions in the text.

Resources

Sentiment Analyses

- **Hutto, C.J. and Gilbert, E.E.**, *VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text*, Eighth International Conference on Weblogs and Social Media (ICWSM-14), Ann Arbor, MI, June 2014.

In our work, the VADER library evaluates sentence sentiment using a lexicon of words with weighted scores, taking into account intensifiers, negations, and context.

- **Bond, Francis and Choo, Merrick**, *Sense and Sentiment*, Proceedings of the Thirteenth Language Resources and Evaluation Conference, Marseille, France, 2022. <https://aclanthology.org/2022.lrec-1.7/>

In our work, the WordNet-based sentiment lexicon determines word sentiment according to word senses.

- **Loria, Steven**, *textblob Documentation*, Release 0.15, 2018.

In our work, the TextBlob library analyzes sentence sentiment based on word-level polarity scores and aggregates them across the sentence.

Entity Analyses

- **Honnibal, Matthew and Montani, Ines**, *spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing*, 2017. To appear in journal.

In our work, spaCy provides models for automatic named entity recognition (NER), including people, organizations, locations, and other categories, and allows tracking of their positions in the text.

3 Approach

The book *Metamorphosis* by Franz Kafka was chosen for its intense emotional content. The entire code was written in Python, as it is a great language for NLP tasks, thanks to its rich ecosystem of libraries and tools specifically designed for text processing and analysis.

Sentiment Analysis

The sentiment analysis begins by loading the text from a file using the `read_document` function, which reads the full content of the text file and returns it as a string for further processing. The text is then cleaned using the `trim_text_edges` function, which removes a fixed number of lines from the beginning and end of the document to exclude non-narrative content.

Since chapters can vary significantly in size, with some being much larger than others, which can skew the analysis results, the text is divided into nearly equal-sized blocks, regardless of chapter boundaries, to mitigate this. The cleaned text is cut into segments of 7,000 characters, as this amount has been identified as an ideal block—neither too large nor too small—for our visualization. This is done using the `safe_block_split` function, which splits the text into blocks of a predefined maximum length while preserving sentence boundaries, ensuring that no sentence is split between blocks. As a result, most blocks are slightly shorter than the maximum size. The block locations are then calculated using the `cumulative_text_lengths` function, which accumulates the lengths of the blocks in words.

Similarly, the text is segmented into chapters using the `split_and_locate_chapters` function, which uses a regular expression to identify chapter boundaries and returns separate chapter segments and their boundary locations.

For both chapters and text blocks, sentiment is computed using three methods implemented by the functions `vader_senti_sentences`, `spacytextblob_senti_sentences`, and `new_wn_senti_sentences`. The WordNet-based method does not consider sentence context like the other two methods; instead, it calculates the average sentiment of all words in a sentence and considers this value the sentence sentiment. In all methods, sentiment is computed for individual sentences, which are treated as atomic units, and then averaged across the corresponding segments (blocks or chapters) to obtain a sentiment score per segment.

Finally, the program aggregates all computed sentiment values into a structured dictionary that associates segment positions (blocks and chapters) with their sentiment scores from all three methods. The results are printed and saved in JSON format using the `save_json` function for further processing or visualization. The results are then visualized to illustrate sentiment progression using the `graph_block_and_chapters_sentiment` function.

Entity Analysis

The program performs a textual analysis of a document by identifying entities and locating them within the text. It begins by reading the text file using the `read_document` function and cleaning it with the `trim_text_edges` function.

Named entities, such as people, organizations, and locations, are automatically extracted using the `find_entities` function, which leverages spaCy's NLP model (vide *Resources*). For each entity, the program returns its category and all positions where it occurs in the text.

In addition to automatic entity recognition, the program allows for manual searches to find positions of unrecognized entities using the `find_words` function.

These two results are then combined, producing a structured mapping of both automatically and manually found entities to their positions in the text. Along with the total word count (important for reference, as positions are relative), the results are printed and saved in JSON format using the `save_json` function, enabling further analysis or visualization.

Finally, the results are visualized using the `plot_entities_positions_from_json` function, allowing the selection of categories to visualize and the exclusion of specific entities. This flexibility ensures the visualization highlights the most relevant data.

4 Results

Sentiment Visualization

In contrast to the original data, the average sentiment for all methods is added to the individual methods and displayed as a red line. The sentiment for the blocks is shown as a curve, while the average sentiment for the chapters is represented by a short horizontal line in the middle of the chapter blocks.

For reference, **Method 1** employs **VADER**, **Method 2** relies on **WordNet**, and **Method 3** makes use of **TextBlob**.

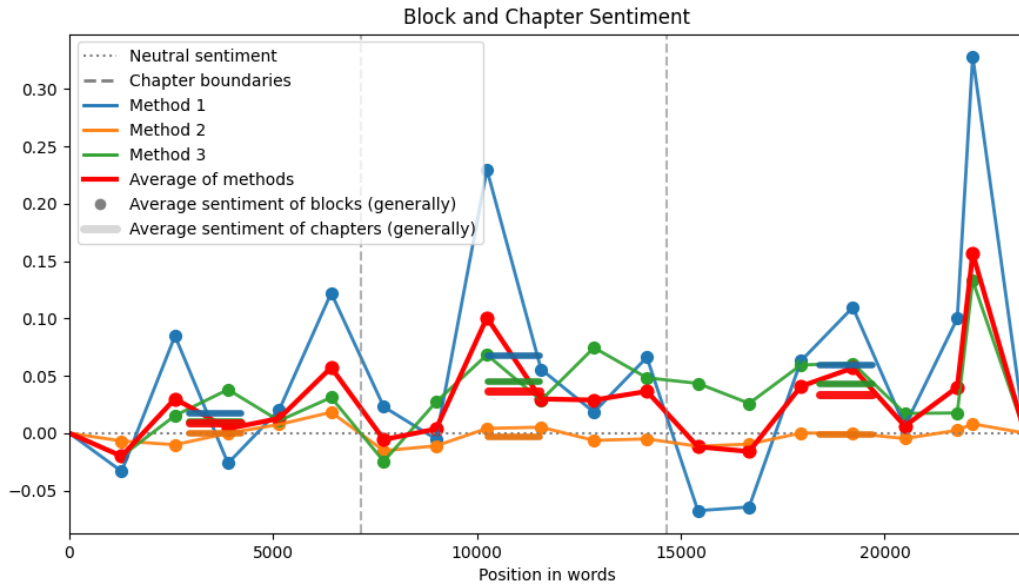


Figure 1: Sentiment progression across blocks and chapters.

Entities Visualization

The same characters are sometimes referred to in different ways: *Mrs. Samsa* as *Mother*, *Mr. Samsa* as *Father*, and *Sister* as *Grete*.

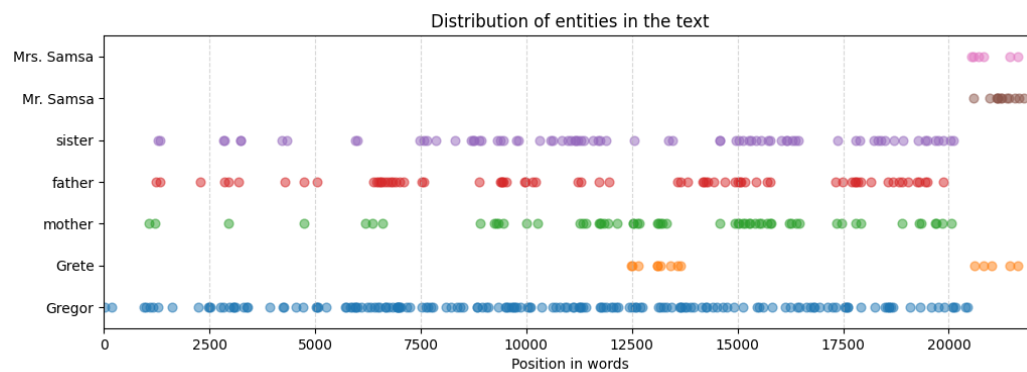


Figure 2: Distribution of named entities across the text.

5 Discussion

Sentiment Visualization

We can observe that the sentiment remains generally neutral, with higher sentiment peaks occurring in five distinct waves. Furthermore, the methods appear to share similar trends, which supports their apparent effectiveness. Overall, we can conclude that the sentiment is slightly positive, which is surprising given the book's typically depressive reputation.

Entities Visualization

The data clearly show that Gregor is the main character, as he appears most frequently and consistently throughout the text. Interestingly, different references to the same person (e.g., *Mrs. Samsa* – *Mother*) don't overlap, which seems to suggest a change in the narrator's relationship with the characters, offering insights into the evolving dynamics between them.

6 Conclusions

In conclusion, this analysis applied natural language processing techniques to assess sentiment and identify named entities within the text. We measured sentiment across both chapters and defined blocks, with results visualized for clearer insights. Named entity recognition mapped the book's characters and locations, which may shed light on character relationships and narrative dynamics. Overall, this work highlights the potential of NLP in deepening literary analysis.